Progettazione di Basi di Dati

La costruzione di una BD si articola generalmente in **tre fasi principali**:

1. **Analisi dei requisiti**
    - Raccolta delle esigenze del committente e degli utenti.
    - Identificazione dei dati da gestire e dei vincoli.
2. **Progettazione**
    - Include le fasi di **modellazione concettuale, logica e fisica**.
    - Si definisce **come rappresentare i dati**, le relazioni e le applicazioni.
3. **Realizzazione**
    - Implementazione pratica sul DBMS scelto.
    - Creazione di tabelle, indici, viste, trigger e applicazioni.

> Nota: spesso si considera l'analisi dei requisiti parte integrante della fase di progettazione.

Ruoli Attorno alla Costruzione di una Base di Dati

1. **Committente**
    - Colui che ha l'esigenza di gestire dati o informazioni.
    - Fornisce i **requisiti e gli obiettivi** della base di dati.
2. **Progettista / Analista**
    - Traduce i requisiti in **progetto concettuale**.
    - Definisce entità, relazioni, vincoli e lo **schema concettuale**.
3. **Programmatore**
    - Realizza la **base di dati sul DBMS**.

- Scrive applicazioni e query che interagiscono con i dati.

4. **Data Base Administrator (DBA)**
    - Gestisce utenti, permessi e sicurezza del sistema.
    - Si occupa di **backup, recovery e ottimizzazione** delle prestazioni.

Progettare una Base di Dati

Significato

Progettare una BD significa preparare in modo strutturato tutto ciò che serve per la sua **realizzazione pratica**.

Attività principali

1. **Definire lo schema globale dei dati**
    - Stabilire quali entità e relazioni esistono nel dominio del discorso.
2. **Definire i vincoli d'integrità**
    - Regole che garantiscono la coerenza e correttezza dei dati (es. chiavi primarie, vincoli referenziali, valori obbligatori).
3. **Definire le operazioni delle applicazioni**
    - Specificare quali query e manipolazioni saranno eseguite dagli utenti o dalle applicazioni.

Importanza

- La progettazione è **necessaria per la fase di realizzazione** della base di dati e delle applicazioni.
- È un'attività **complessa e strategica**, per la quale sono state sviluppate molte **metodologie e ambienti di supporto**.

Modellare una Base di Dati

- La **modellazione** è fondamentale per progettare una BD efficiente e coerente.
- Obiettivo: i dati devono essere un **modello fedele del dominio del discorso** (la realtà o il contesto che vogliamo rappresentare).
- La modellazione permette di rappresentare il dominio **in modo astratto o simbolico**, facilitando:
  - la comprensione dei dati,
  - la definizione di entità e relazioni,
  - la progettazione dello schema concettuale.

Fasi della Progettazione di una Base di Dati

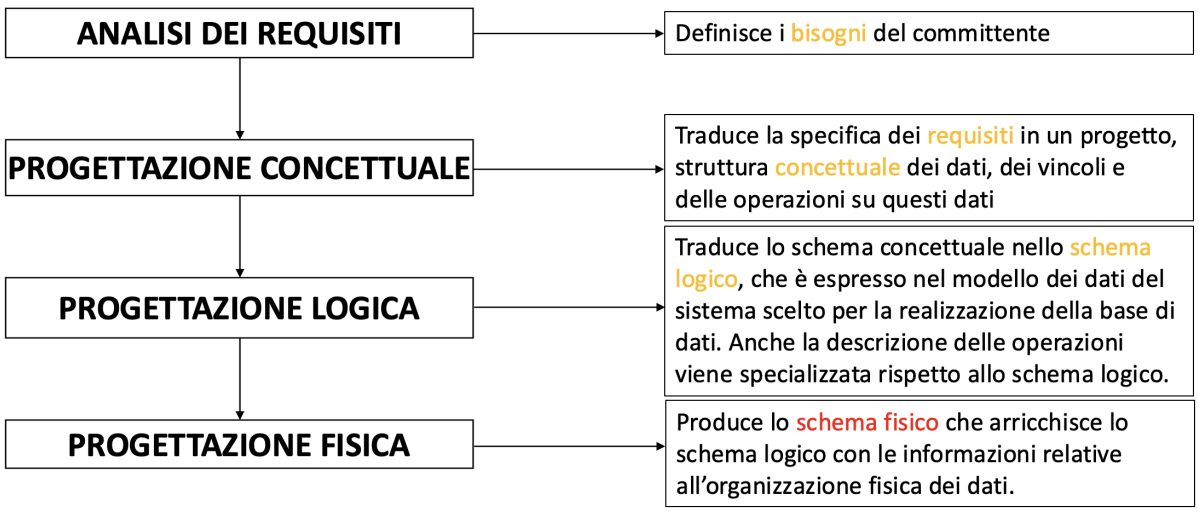
Ogni fase è centrata sulla **modellazione dei dati**, cioè sulla rappresentazione astratta del dominio del discorso.



1. **Analisi dei requisiti**
   - Raccolta delle **esigenze del committente** e degli utenti.
   - Identificazione dei **dati da gestire**, delle informazioni da produrre e dei vincoli.
2. **Progettazione concettuale**
   - Creazione dello **schema concettuale** basato sull'analisi dei requisiti.

- Rappresentazione tramite **modelli astratti**, tipicamente **Entity-Relationship (ER)**.
- Definizione di entità, relazioni, attributi e vincoli.

3. **Progettazione logica**
   - Traduzione dello schema concettuale in **schema logico** adatto a un modello di dati specifico (es. relazionale).
   - Scelta di **tabelle, attributi, chiavi primarie e chiavi esterne**.
4. **Progettazione fisica**
   - Definizione di come i dati saranno **memorizzati fisicamente** nel DBMS.
   - Scelta di **indici, strutture di memorizzazione, partizionamenti** e ottimizzazioni per le prestazioni.





Aspetti del Problema nella Modellazione di una Base di Dati

Quando si costruisce un modello di dati, è utile distinguere diversi **aspetti**:

1. **Aspetto ontologico**
   - Si riferisce alla **conoscenza del dominio del discorso**.
   - Studia ciò che **esiste o si suppone esistere** nell'universo del discorso e quindi deve essere modellato.
2. **Aspetto logico**
   - Riguarda i **meccanismi di astrazione** utilizzati per modellare la realtà.
   - Esprime come entità, relazioni e vincoli vengono formalizzati.
3. **Aspetto linguistico**
   - Si riferisce al **linguaggio formale** utilizzato per definire il modello.
   - Esempio: diagrammi ER, modelli relazionali, linguaggi di specifica.

4. **Aspetto pragmatico**
    - Definisce la **metodologia** da seguire nel processo di modellazione.
    - Comprende regole e procedure finalizzate alla costruzione del modello informatico.

Aspetto Ontologico: Cosa si Modella

Quando si decide **cosa modellare** in una BD, si distinguono diversi tipi di conoscenza:

1. **Conoscenza concreta**
    - I **fatti** effettivamente esistenti nel dominio del discorso.
    - Esempio: uno studente con matricola 080709 ha sostenuto l'esame di Basi di Dati.
2. **Conoscenza astratta**
    - La **struttura e i vincoli** che regolano i fatti concreti.
    - Esempio: ogni studente può sostenere più esami, ma un esame può avere solo un voto per studente.
3. **Conoscenza procedurale / comunicazioni**
    - Le **operazioni sugli oggetti** e le modalità di interazione con il sistema.
    - Esempio: query per estrarre gli studenti con voto ≥ 28, aggiornamenti sui voti, gestione utenti.

> Nel seguito della progettazione, l'attenzione si concentra soprattutto sulla conoscenza concreta e astratta, cioè sui dati e sulle regole che li governano.

Conoscenza Concreta

- Riguarda i **fatti specifici** da rappresentare in una base di dati.
- Include:

1. **Entità**: oggetti di interesse di cui vogliamo rappresentare fatti o caratteristiche.
2. **Collezioni di entità omogenee**: insiemi di entità dello stesso tipo (es. tutti i libri della biblioteca).
3. **Associazioni fra entità**: relazioni tra entità (es. prestito di un libro da parte di un utente).

Entità e Proprietà

1. **Entità**
   - Sono ciò di cui vogliamo rappresentare **alcuni fatti o proprietà**.
   - Esempi: libro, prestito, utente della biblioteca, modello di elettrodomestico.
2. **Proprietà (Attributi)**
   - Fatti che **descrivono caratteristiche** di un'entità.
   - Esempio: tipo di elettrodomestico, peso, prezzo, marca.
3. **Punti chiave**
   - I valori delle proprietà **caratterizzano l'entità**, ma non definiscono l'entità stessa.
   - Un'entità può cambiare valore delle proprietà nel tempo, ma resta la stessa entità.
   - Due entità distinte possono avere **stesse proprietà e stessi valori** (es. due persone con lo stesso nome e età).

Esempio pratico

Catena di negozi che vende elettrodomestici:

- **Entità**: Modello di elettrodomestico, Centro di assistenza
- **Proprietà di "Modello di elettrodomestico"**: tipo, peso, prezzo, marca

Proprietà (o Attributi)

- Una **proprietà** è una **coppia**:

  (*nomeattributo*,*valorediuncertotipo*)(nome attributo, valore di un certo tipo) (*nomeattributo*,*valorediuncertotipo*)

  - Esempio: (Nome, “Riccardo”), (Nome, “Giovanna”)
- Ogni proprietà ha un **dominio**, cioè l’insieme dei possibili valori che può assumere.

Classificazione delle proprietà

1. **Atomica vs Strutturata**
   - Atomica: il valore non è ulteriormente scomponibile (es. età).
   - Strutturata: il valore può essere scomposto in più elementi (es. indirizzo: via, città, CAP).
2. **Univoca vs Multivalore**
   - Univoca: ogni entità ha al massimo un valore per la proprietà (es. codice fiscale).
   - Multivalore: un’entità può avere più valori (es. numeri di telefono).
3. **Totale (obbligatoria) vs Parziale (opzionale)**
   - Totale: tutte le entità devono avere un valore specificato.
   - Parziale: alcune entità possono non avere un valore.
4. **Costante vs Variabile**
   - Costante: il valore non cambia nel tempo.
   - Variabile: il valore può cambiare (es. indirizzo, saldo di un conto).
5. **Calcolata vs Non calcolata**
   - Calcolata: il valore può essere derivato da altre proprietà (es. età calcolata dalla data di nascita).

- Non calcolata: il valore deve essere memorizzato esplicitamente.

Tipo di Entità

- Un **tipo di entità** è una **descrizione astratta** di ciò che accomuna un insieme di entità omogenee, esistenti o possibili.
- Non è una collezione di entità esistenti, ma definisce la **natura e le caratteristiche** di tutte le entità di quel tipo.

Esempio

- Entità: Riccardo, Giovanna
- Tipo di entità: **Persona**
- Proprietà: Nome (string)
- Significato: il tipo Persona descrive **tutte le persone possibili**, presenti o future, e non solo quelle già esistenti.

Caratteristiche principali

- È **astratto** e generale.
- Rappresenta un insieme teoricamente **infinito** di entità possibili.
- Definisce le **proprietà comuni** delle entità del tipo.

Esempi di Tipi di Entità e loro Proprietà

Tipo di Entità	Proprietà
Studente	Nome, AnnoNascita, Matricola, E-mail, ...
Esame	Materia, Candidato, Voto, Lode, Data, ...
Auto	Modello, Colore, Cilindrata, Targa, ...
Descrizione bibliografica	Autori, Titolo, Editore, LuogoEdizione, ...

Scelta delle Entità e delle Proprietà

- La distinzione tra **entità** e **proprietà** dipende dal **contesto** e dall'interpretazione dei fatti.
- Alcuni fatti possono essere modellati come **proprietà** in un contesto e come **entità** in un altro.

Esempi pratici

1. **Descrizione bibliografica come entità unica**
    - Proprietà: Autori, Titolo, Editore, LuogoEdizione, Anno
2. **Descrizione bibliografica separata da altre entità**
    - Entità:
        - ***Descrizione bibliografica***: Titolo, Anno
        - ***Autore***: Nome, Nazionalità, AnnoNascita
        - ***Editore***: Nome, LuogoEdizione, E-mail

> La modellazione deve riflettere la struttura concettuale più significativa del dominio e come si vuole rappresentare e manipolare l'informazione.

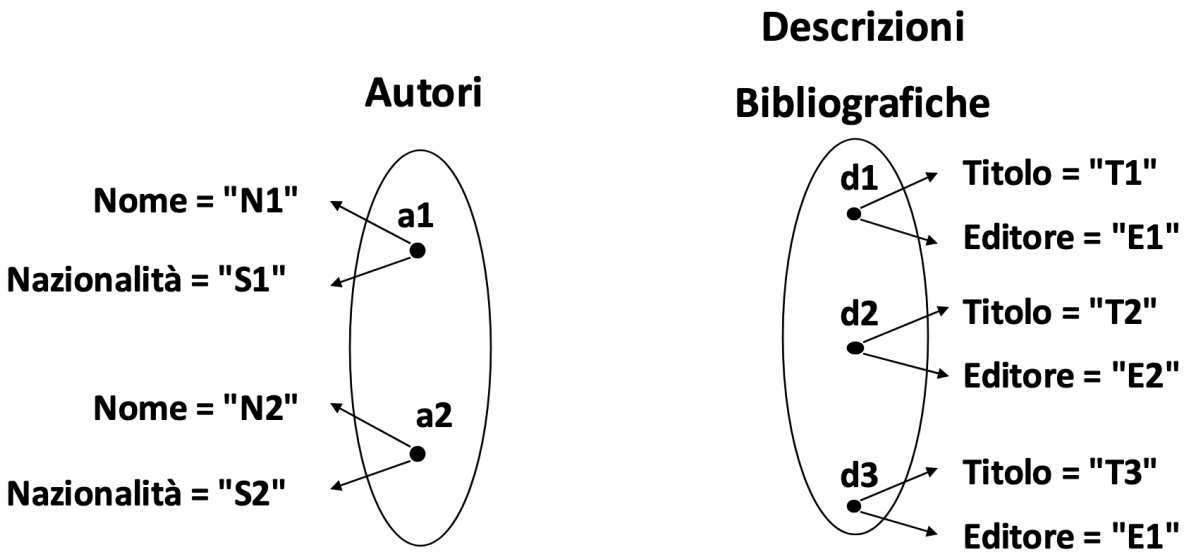
Collezione

- Una **collezione** è un insieme **variabile nel tempo** di entità **omogenee** (stesso tipo) che sono rilevanti per il dominio del discorso.
- L'**insieme degli elementi di una collezione in un momento specifico** è detto **estensione della collezione**.

Differenze tra Tipo e Collezione

Concetto	Descrizione
Tipo di entità	Definisce tutte le possibili entità di quel tipo, infinite e teoriche

Concetto	Descrizione
Collezione	Insieme finito e variabile di entità di quel tipo, realmente presenti o considerate nel modello



Associazioni

- Un'**istanza di associazione** è un fatto che **collega due o più entità**, creando un legame logico tra di loro.

Esempi

- La descrizione bibliografica con titolo "Basi di Dati" riguarda il documento fisico con collocazione "d3-55-2".
- L'utente Tizio ha in prestito una copia della **Divina Commedia**.

Definizione formale

- Un'associazione $R(X, Y)$ fra due collezioni di entità X e Y è un insieme di **istanze di associazione** tra elementi di X e Y, che può **variare nel tempo**.
- Il **prodotto cartesiano** $X \times Y$ è detto **dominio dell'associazione**.

Proprietà strutturali

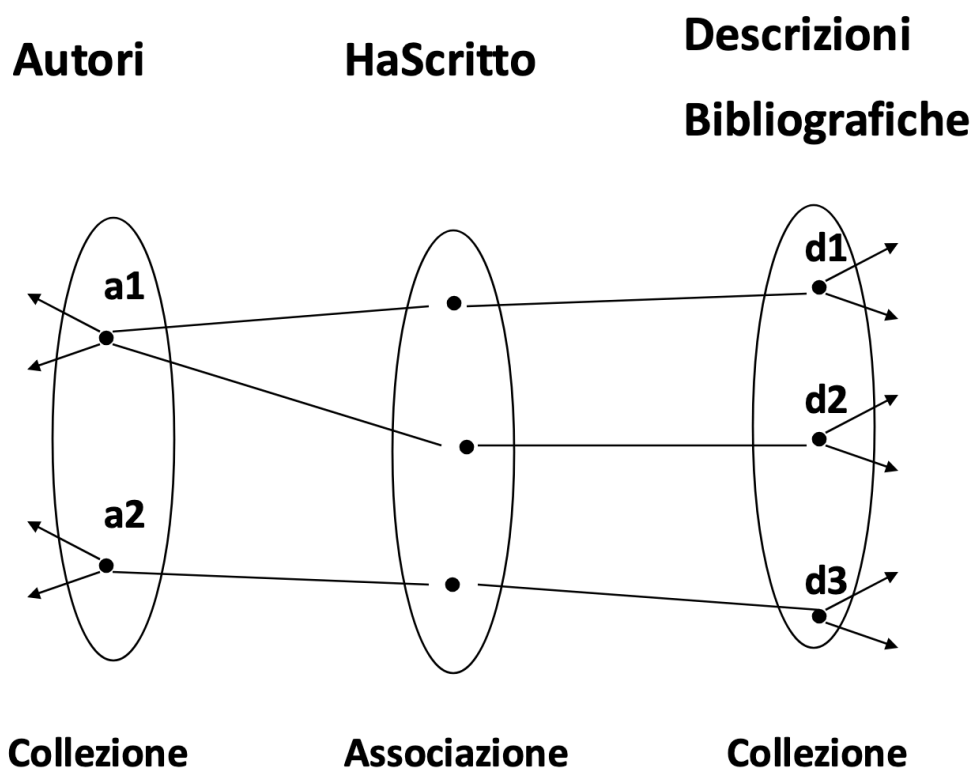
1. **Molteplicità**: quante istanze di una collezione possono essere associate a un'istanza dell'altra collezione.
2. **Totalità**: se tutte le entità di una collezione partecipano all'associazione oppure solo alcune.

Associazioni come Relazioni Matematiche

- Se consideriamo due collezioni X e Y come due insiemi, un'**istanza di associazione** tra X e Y può essere rappresentata come una **coppia ordinata** (x, y) con $x \in X, y \in Y$
- L'**associazione** R tra X e Y può quindi essere vista come un **sottoinsieme del prodotto cartesiano** $X \times Y$

  $R \subseteq X \times Y$
- In altre parole, un'associazione tra collezioni è una **relazione nel senso matematico**, costituita da tutte le coppie (x, y) che verificano il legame logico tra le entità.

> Questo formalismo permette di gestire associazioni complesse in modo chiaro e rigoroso nella progettazione di basi di dati.



Tipi di Associazione – Molteplicità

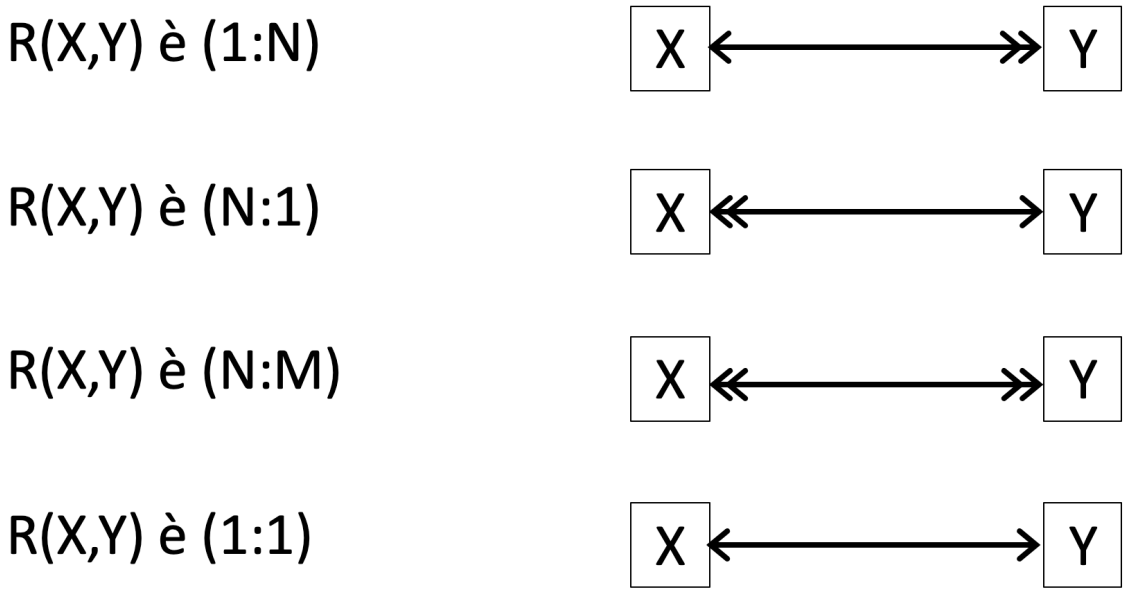
Vincolo di Univocità

- Un'associazione $R(X, Y)$ è **univoca rispetto a** X se per ogni elemento $x \in X$ esiste al **più un elemento** $y \in Y$ associato a x.
- Se non vale questo vincolo, l'associazione è **multivalore rispetto a** X.

Cardinalità di un'associazione

Tipo	Significato
$1 : N$	Multivalore su X, univoca su Y → uno-a-molti
$N : 1$	Univoca su X, multivalore su Y → molti-a-uno

Tipo	Significato
$N : M$	Multivalore su X e multivalore su Y → molti-a-molti
$1 : 1$	Univoca su X e univoca su Y → uno-a-uno



Esempi di Associazioni tra Collezioni

Associazione	Collezioni coinvolte	Molteplicità / Tipo
Frequenta	Studenti → Corsi	Uno studente può frequentare molti corsi → ($1 : N$)
Insegna	Professori → Corsi	Un professore può insegnare molti corsi, ogni corso è insegnato da un solo professore → ($1 : N$)
SuperatoDa	Esami → Studenti	Un esame è superato da uno studente, uno studente può superare molti esami → ($N : 1$)

Associazione	Collezioni coinvolte	Molteplicità / Tipo
Dirige	Professori → Dipartimenti	Un professore dirige un dipartimento, un dipartimento può avere molti professori? → tipicamente ($1:1$) o ($1:N$) a seconda del modello

> Questi esempi mostrano come la **molteplicità** definisca i vincoli tra le entità nella progettazione concettuale delle basi di dati, fornendo le basi per la successiva implementazione in un DBMS.

Tipi di Associazione – Totalità

Vincolo di Totalità

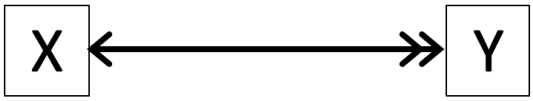
- Un'associazione $R(X, Y)$ è **totale (o suriettiva) su** X se **ogni elemento** $x \in X$ è associato **ad almeno un elemento** $y \in Y$.
- Se questo vincolo non vale, l'associazione è **parziale rispetto a** X.

Esempio

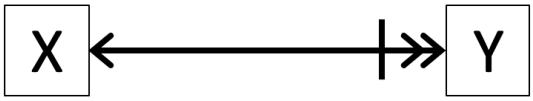
- **Insegna(Professori, Corsi)** è totale su **Corsi** perché **ogni corso deve avere almeno un docente** che lo insegna.

> La totalità indica quindi se la partecipazione di tutti gli elementi di una collezione è obbligatoria nell'associazione, fornendo un vincolo fondamentale per il modello concettuale della BD.

R(X,Y) è totale



R(X,Y) è parziale



Un'associazione tra due collezioni, C_1 e C_2, viene rappresentata tramite **una linea che unisce le due collezioni**.

- La linea è **etichettata con il nome dell'associazione**, solitamente formulato come un predicato ("soggetto predicato complemento").
- L'**univocità** dell'associazione rispetto a C_1 si indica con **una freccia singola** dalla collezione C_1 verso C_2; se non esiste vincolo di univocità, si usa **una freccia doppia**.
- La **parzialità** è rappresentata con **un taglio sulla linea vicino alla freccia**, mentre la **totalità** si rappresenta con **l'assenza di taglio**.

Tipi di Associazione – Persone e Città

Associazione	Cardinalità	Totalità / Parzialità
NataA(Persone, Città)	$N : 1$	Totale su Persone, Parziale su Città
HaVisitato(Persone, Città)	$N : M$	Parziale su Persone e Città
ÈSindacoDi(Persone, Città)	$1 : 1$	Parziale su Persone, Totale su Città
ÈVissutaA(Persone, Città)	$N : M$	Totale su Persone e Città

> Questo esempio mostra come le associazioni possano avere **diverse combinazioni di molteplicità e vincoli di totalità/parzialità**, fondamentali per modellare correttamente le relazioni tra entità in un database.

Struttura della Conoscenza Concreta

La conoscenza concreta si organizza intorno a due elementi principali:

1. **Collezioni**
    - Esistenza delle collezioni rilevanti nel dominio.
    - Nome di ciascuna collezione.
    - Tipo degli elementi contenuti nelle collezioni, comprensivo di:
        - Nome delle proprietà degli elementi
        - Caratteristiche e vincoli delle proprietà
2. **Associazioni**
    - Esistenza delle associazioni tra collezioni.
    - Nome di ciascuna associazione.
    - Collezioni coinvolte nell'associazione.
    - Proprietà strutturali dell'associazione, come:
        - Molteplicità ($1:1$, $1:N$, $N:1$, $N:M$)
        - Vincoli di totalità o parzialità

> In sintesi, la struttura della conoscenza concreta definisce quali entità e collezioni esistono, come sono caratterizzate e come sono correlate tra loro.

Conoscenza Astratta

La conoscenza astratta riguarda i **fatti generali** che descrivono e regolano la conoscenza concreta. Include:

1. **Struttura della conoscenza concreta**
    - Collezioni
    - Tipi di entità

- Associazioni

2. **Vincoli sulla conoscenza concreta**
   - Limitazioni sui valori possibili delle proprietà
   - Regole su come i dati possono evolvere nel tempo
3. **Tipi di vincoli**
   - **Vincoli statici**: condizioni sempre valide sulle entità e associazioni
   - **Vincoli dinamici**: condizioni che riguardano l'evoluzione dei dati nel tempo
4. **Regole di derivazione**
   - Permettono di ottenere nuovi fatti a partire da fatti già noti

> In sintesi, la conoscenza astratta serve a controllare, vincolare e derivare informazioni sulla struttura e l'evoluzione dei dati nel database.

Aspetto Logico: Modello dei Dati a Oggetti

1. **Definizione**

   Un **modello dei dati** fornisce meccanismi di astrazione per descrivere la **struttura della conoscenza concreta**.

2. **Schema**

   Lo **schema** rappresenta la struttura dei dati e i vincoli di integrità, utilizzando un particolare modello dei dati.

3. **Notazione grafica**

   Per la rappresentazione si possono usare varianti dei **diagrammi a oggetti** o dei **diagrammi ER (Entità-Relazione)**.

4. **Nozioni fondamentali**
   - **Oggetto**: entità concreta con proprietà e comportamento
   - **Tipo di oggetto / Classe**: descrizione astratta delle entità omogenee

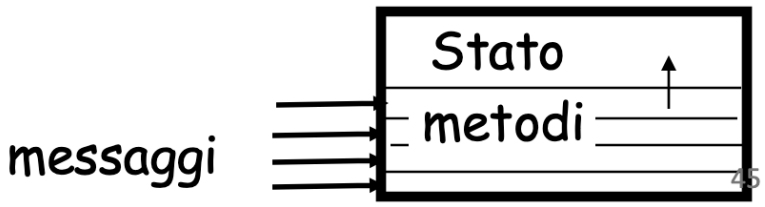
- **Ereditarietà**: possibilità per un tipo di oggetto di derivare caratteristiche da un altro
- **Gerarchia fra tipi e fra classi**: organizzazione dei tipi o classi in strutture gerarchiche

> Questo approccio permette di modellare la conoscenza concreta in modo più vicino alla realtà, con un occhio alla riusabilità e alla struttura logica dei dati.

Gli Oggetti nel Modello a Oggetti

1. **Definizione**
   - Ad ogni entità del dominio corrisponde un **oggetto** nel modello.
   - Un oggetto è un'entità software con **stato**, **comportamento** e **identità** che rappresenta un'entità reale.
2. **Stato**
   - Rappresentato da un insieme di **costanti o variabili**.
   - I valori possono avere qualsiasi livello di complessità.
3. **Comportamento**
   - Definito da **procedure locali** chiamate **metodi**.
   - I metodi modellano le operazioni di base sull'oggetto e sulle proprietà derivabili.
4. **Interazione**
   - Un oggetto può ricevere **messaggi**.
   - Risponde restituendo valori **memorizzati nello stato** o **calcolati tramite metodi**.

> Gli oggetti permettono di modellare le entità del dominio in modo dinamico, con capacità di memorizzare dati e di reagire a richieste tramite comportamenti predefiniti.



Il Tipo Oggetto

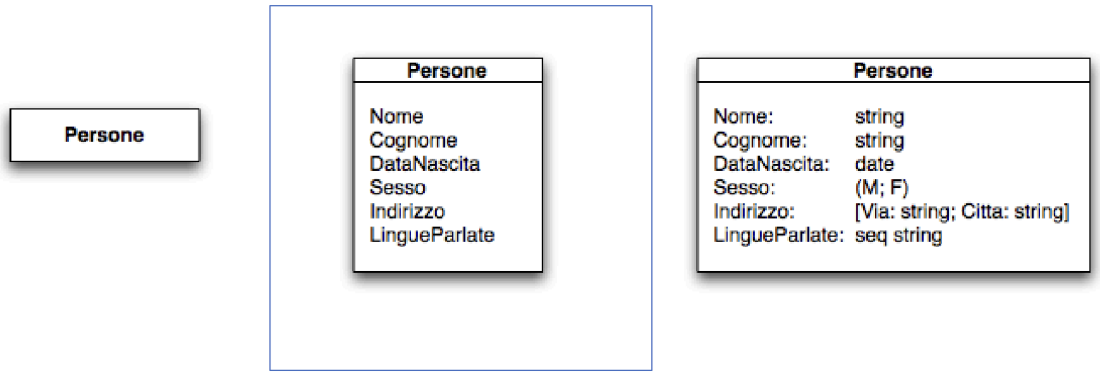
1. **Definizione**
    - Un **tipo oggetto** definisce l'insieme dei **messaggi** (cioè l'interfaccia) a cui possono rispondere tutti gli oggetti appartenenti a quel tipo.
    - I **nomi dei messaggi** sono anche considerati **attributi** dell'oggetto.
2. **Rappresentazione nei diagrammi ER**
    - I **tipi oggetti** non sono rappresentati esplicitamente nei diagrammi ER, poiché l'attenzione è sulle **collezioni** e sulle **associazioni**.
    - Tuttavia, la **collezione** grafica indica anche gli **attributi** del tipo oggetto associato.

> In pratica, il tipo oggetto definisce l'interfaccia e le proprietà comuni degli oggetti, mentre nei diagrammi ER l'attenzione resta sulla struttura e le relazioni fra collezioni.

Le Classi

- Una **classe** è un insieme di **oggetti dello stesso tipo**.
- La classe può essere **modificata** tramite operatori che permettono di **includere** o **estrarre** oggetti dall'insieme.

> In sintesi, una classe raggruppa oggetti omogenei e fornisce un contesto per gestirli collettivamente.



Una classe Persone a diversi livelli di specifica

Esempio: Associazione fra Studenti ed Esami

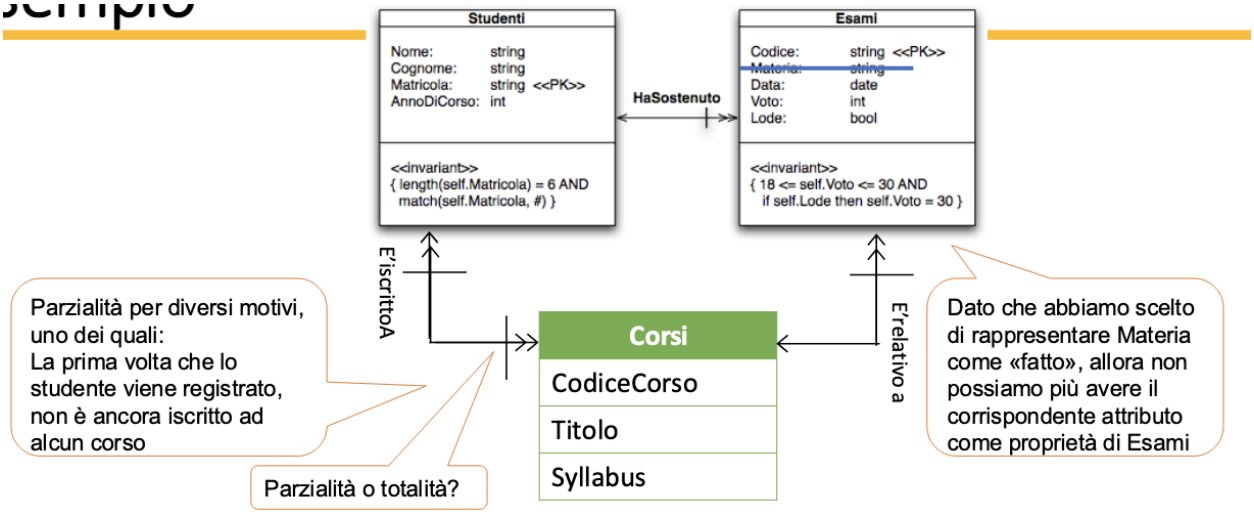
- **Scenario**: Rappresentiamo le classi coinvolte nella gestione degli esami degli studenti, considerando ogni esame come un **evento di esame superato** (registrato).
- **Vincoli sull'associazione**:
  1. Ogni esame riguarda **uno ed un solo studente** → univocità (freccia singola) e totalità (assenza del taglio) sul lato studente.
  2. Uno studente può avere **molti esami superati** → parzialità (taglio) e assenza di univocità (freccia doppia) sul lato esami.



Le Associazioni: *Esempio con Esami*

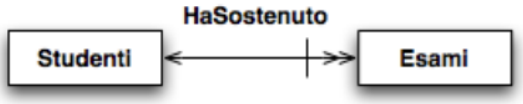
- **Esami**: rappresentano gli **eventi degli esami** (istanze di verbalizzazione).
- **Vincoli dell'associazione Studente–Esame**:
  1. Uno **studente** può sostenere **zero o più esami**.
  2. Un **esame** riguarda **solo uno studente**.
- **Errore comune**: inserire i dettagli dello studente (Matricola, Nome, Cognome, ecc.) come proprietà dell'esame.
- **Nota**: In questo esempio, **Materia** è un attributo di Esami, non dello studente.

> L'associazione corretta separa chiaramente entità (Studente) e evento (Esame), evitando ridondanza e mantenendo la coerenza del modello.

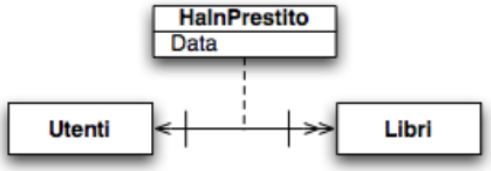


Le Associazioni

- Le associazioni si modellano con un **costrutto apposito** nei diagrammi.



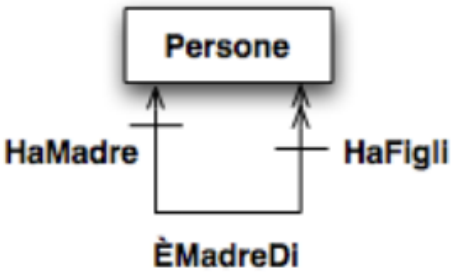
- Possono avere **proprietà proprie** (es. data di inizio di un'associazione "lavora_in").



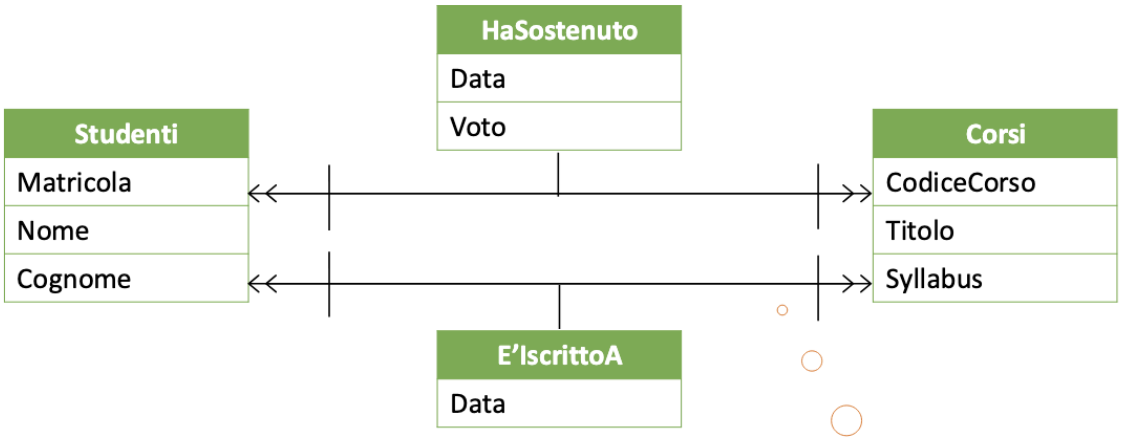
- Possono essere **ricorsive**, cioè riguardare elementi della **stessa collezione**.

Associazioni Ricorsive

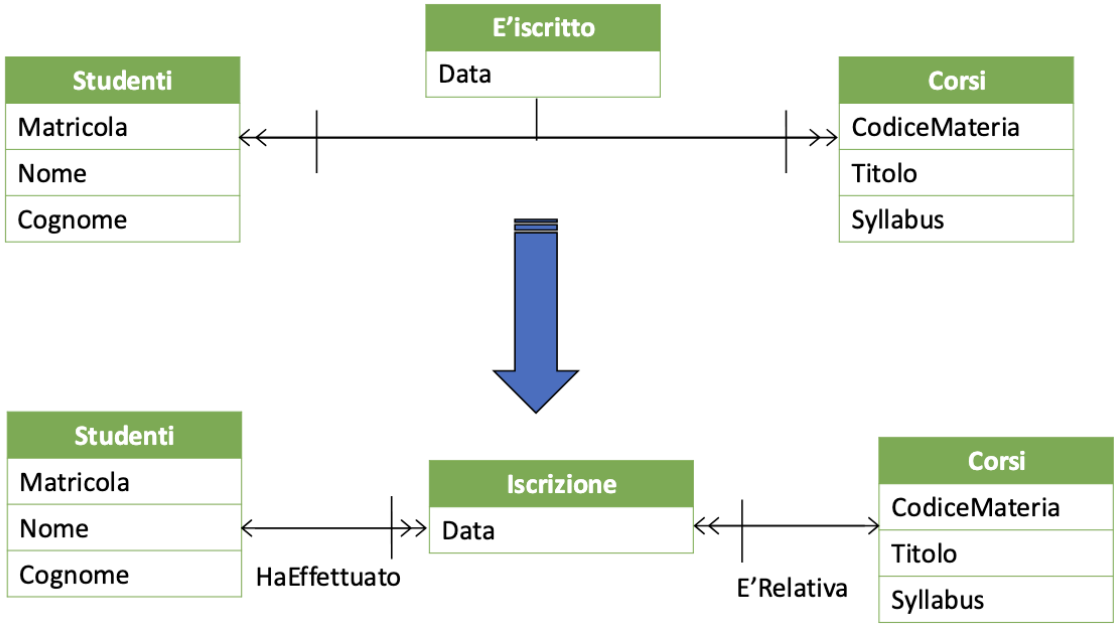
- Sono **relazioni binarie tra elementi della stessa collezione**.
- Esempio: una persona può essere **madre** di più persone della collezione e può avere a sua volta una **madre** nella stessa collezione.



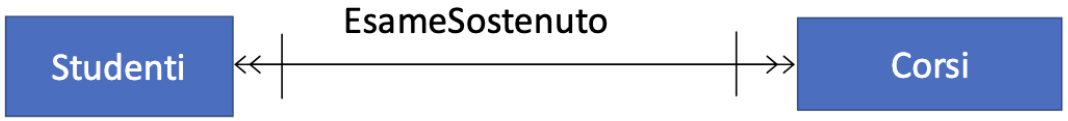
- Per rappresentarle correttamente:
  - Etichettare la linea con il **nome dell'associazione**.
  - Specificare i **ruoli** dei due componenti in ogni istanza dell'associazione (es. "madre" e "figlio").

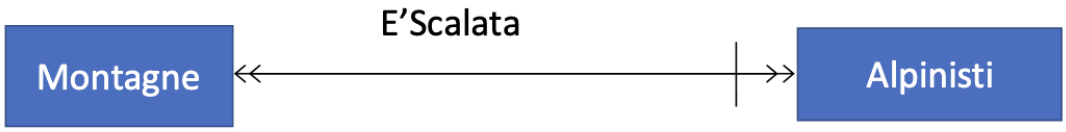


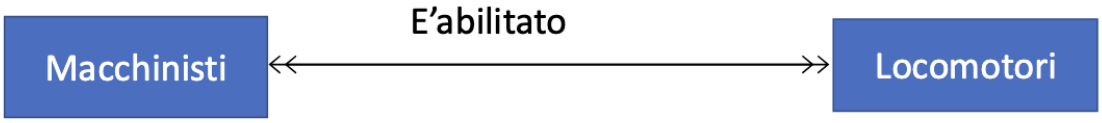
Esempio: *Reificazione delle molti a molti*



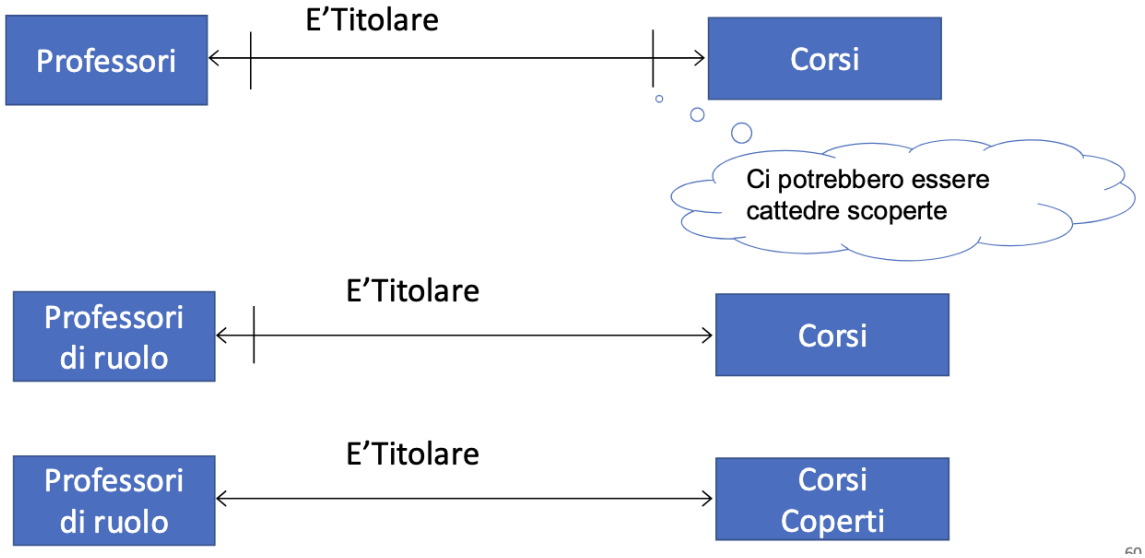
Esempio: *Molti a molti*





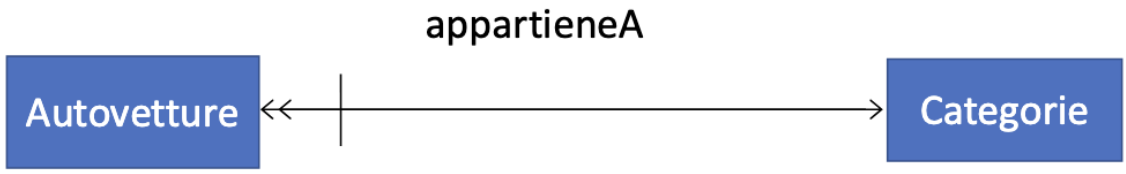


Esempio: *Uno a uno*



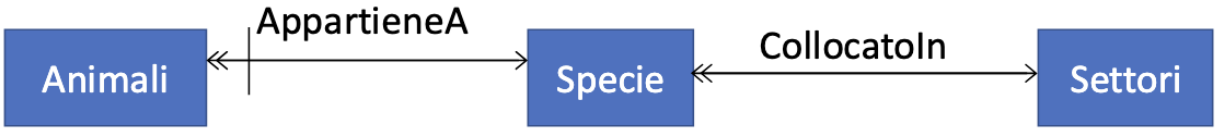
Esercizio: *Agenzia Noleggio*

Una agenzia di noleggio di autovetture ha un parco macchine ognuna delle quali ha una targa, un colore e fa parte di una categoria; per ogni categoria c'è una tariffa di noleggio.



Esercizio: *Zoo*

In un giardino zoologico ci sono degli animali appartenenti a una specie e aventi una certa età; ogni specie è localizzata in un settore (avente un nome) dello zoo.



Esercizio: *Casa Discografica*

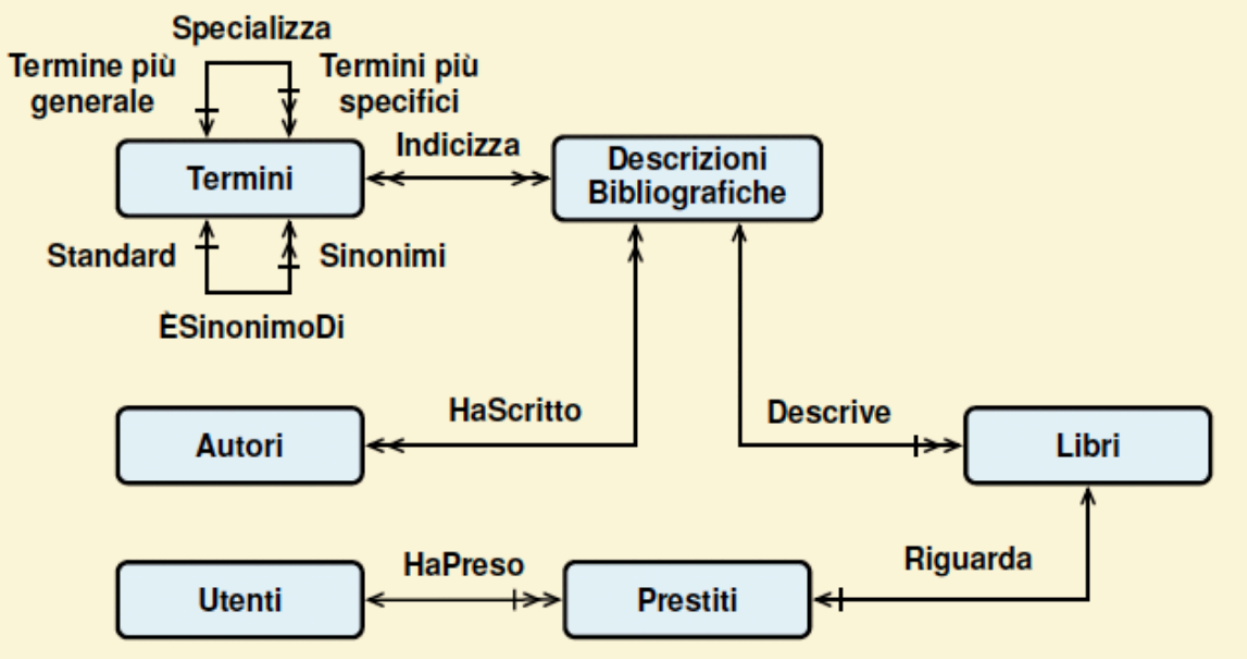
Una casa discografica produce dischi aventi un codice e un titolo; ogni disco è inciso da uno o più cantanti, ognuno dei quali ha un nome, un indirizzo e, qualcuno, un nome d'arte.



Esercizio: *Biblioteca Universitaria*

Si vogliono modellare alcuni fatti riguardanti una biblioteca universitaria:

- le descrizioni bibliografiche dei libri, opere con un solo volume,
- i termini del thesaurus (parole chiave),
- le copie dei libri disponibili che corrispondono ad una descrizione bibliografica,
- gli autori dei libri,
- gli utenti della biblioteca
- i prestiti in corso.



Gerarchie di Classi

- Le **classi di entità** possono essere organizzate in una **gerarchia di specializzazione/generalizzazione**.

- Una classe che si trova **a un livello inferiore** rispetto ad altre viene detta **sottoclasse**, mentre le classi superiori sono dette **superclassi**.

Caratteristiche principali delle gerarchie

1. **Ereditarietà delle proprietà**:
    - Le sottoclassi ereditano gli attributi e le associazioni definite nelle superclassi.
2. **Inclusione degli elementi**:
    - Gli elementi di una sottoclasse costituiscono un **sottoinsieme** degli elementi della superclasse.

> Questo meccanismo permette di riutilizzare la definizione delle proprietà e di organizzare le entità secondo livelli di astrazione più specifici.

Scelta delle Sottoclassi

Esempio 1: Studenti universitari

La classe **Studenti universitari** può essere generalizzata in diverse sottoclassi a seconda del contesto:

- **Matricole** e **Laureandi**
- **Studenti in corso** e **Studenti fuori corso**
- **Studenti pisani** e **Studenti fuori sede**

Esempio 2: Persone

La classe **Persone** può essere generalizzata in:

- **Patentati** (con specifica del tipo di patente) e **Non patentati**
- **Studenti** e **Non studenti**

> La scelta delle sottoclassi dipende dalle proprietà e dagli obiettivi informativi che si vogliono rappresentare.

Gerarchia tra Tipi di Oggetto

Tra i **tipi oggetto** esiste una relazione di **sottotipo** con le seguenti caratteristiche:

Proprietà della relazione di sottotipo

- **Asimmetrica, riflessiva e transitiva** → quindi è una relazione di **ordine parziale**.
- **Sostituibilità (Liskov Substitution Principle)**: se T è sottotipo di T′, allora gli elementi di T possono essere usati ovunque siano richiesti valori di tipo T′.

Conseguenze pratiche

- Gli elementi di T **ereditano tutte le proprietà** degli elementi di T′.
- Per ogni proprietà ppp definita in T′, il suo tipo in T è un **sottotipo** del tipo in T′.

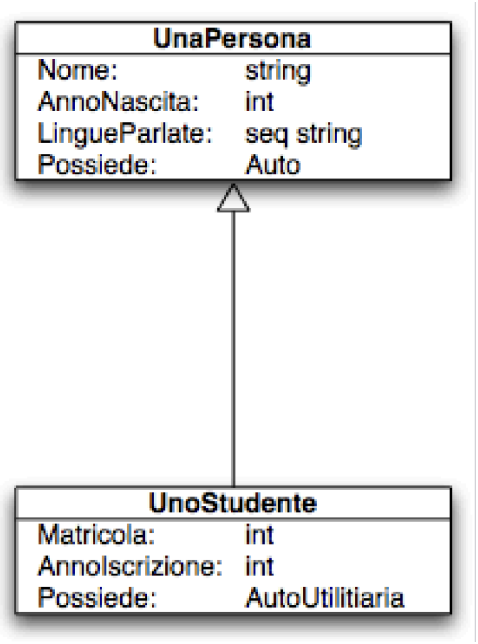
Tipi di gerarchia

- **Semplice**: un tipo ha al massimo una superclasse.
- **Multipla**: un tipo può avere più superclassi.

Ereditarietà (Inheritance)

L'**ereditarietà** permette di definire:

1. Un **tipo oggetto** a partire da un altro tipo oggetto.
2. L'**implementazione** di un tipo oggetto a partire dall'implementazione di un altro.



Uso tipico

- L'**ereditarietà tra tipi** serve per creare **sottotipi**.
- L'**ereditarietà tra implementazioni** serve per definire le implementazioni dei sottotipi (**ereditarietà stretta**).

Regole nell'ereditarietà stretta

- Gli **attributi possono essere aggiunti**, ma non rimossi.
- Gli **attributi esistenti possono essere ridefiniti** solo **specializzandone il tipo**.

Gerarchia tra Classi

Fra le classi può essere definita una relazione di **sottoclasse** (o sottoinsieme) con queste proprietà:

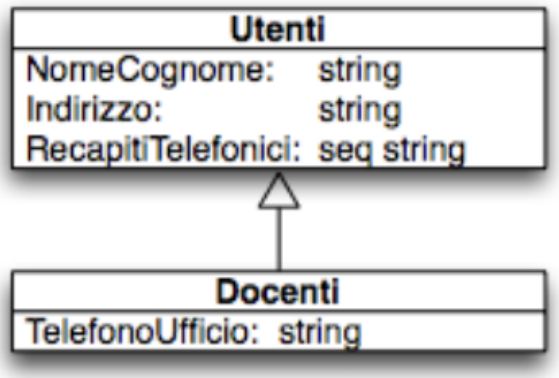
1. La relazione è **asimmetrica, riflessiva e transitiva**.
2. **Vincolo intensionale:** Se C è sottoclasse di C', allora il **tipo degli elementi di** C è sottotipo del tipo degli elementi di C'

3. **Vincolo estensionale:** Se C è sottoclasse di C', allora gli **elementi di** C sono un sottoinsieme degli elementi di C'

Tipi di Gerarchie e Vincoli

Inclusione

- Le sottoclassi sono **sottoinsiemi** della superclasse.

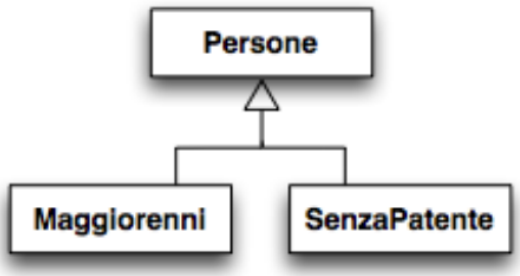


Vincoli su insiemi di sottoclassi

1. **Disgiunzione**: due sottoclassi non condividono elementi (insiemi mutuamente esclusivi).
2. **Copertura**: l'insieme delle sottoclassi **copre tutta la superclasse**; ogni elemento della superclasse appartiene ad almeno una sottoclasse.

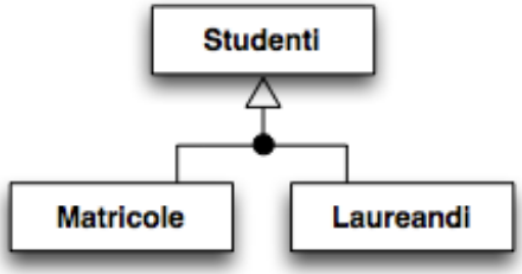
- **Sottoclassi scorrelate**
  - Le sottoclassi non hanno vincoli tra loro.
  - Gli elementi di una sottoclasse possono o non possono appartenere ad altre sottoclassi.





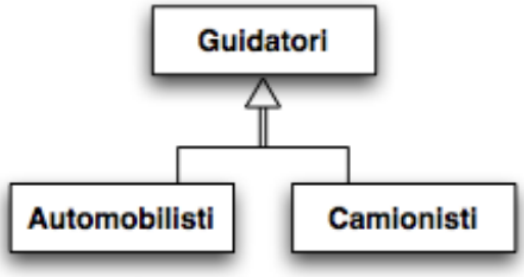
Sottoclassi scorrelate

- **Sottoclassi disgiunte**
  - Nessun elemento appartiene a più di una sottoclasse.
  - Vincolo di **mutua esclusione**.



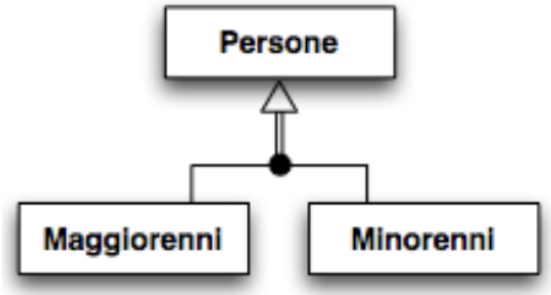
Sottoclassi disgiunte

- **Sottoclassi coperture**
  - Ogni elemento della superclasse appartiene ad almeno una sottoclasse.
  - Vincolo di **copertura totale**, ma non necessariamente disgiunte.



Sottoclassi copertura

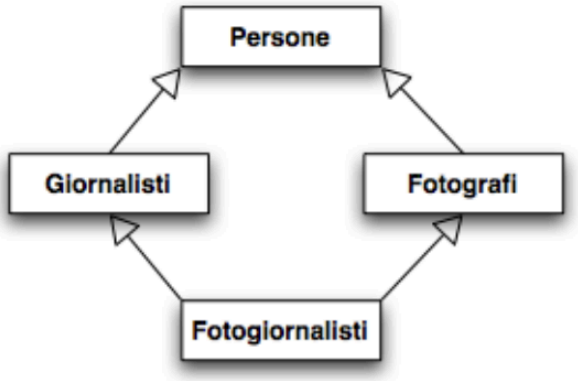
- **Sottoclassi partizione**
  - Combinano disgiunzione e copertura totale.
  - Ogni elemento della superclasse appartiene a **una e una sola** sottoclasse.



Sottoclassi partizione

Gerarchia Multipla

- **Ereditarietà singola**: un tipo oggetto eredita proprietà e metodi da un solo supertipo.
- **Ereditarietà multipla**: un tipo oggetto può ereditare da più supertipi contemporaneamente.



Vantaggi:

- Consente di combinare funzionalità e attributi da più supertipi.

Problemi possibili:

- **Conflitti di attributi/metodi**: lo stesso attributo o metodo può essere ereditato da più supertipi con tipi o implementazioni differenti, generando ambiguità.
- Necessità di meccanismi per risolvere i conflitti (ad esempio rinomina o priorità tra supertipi).

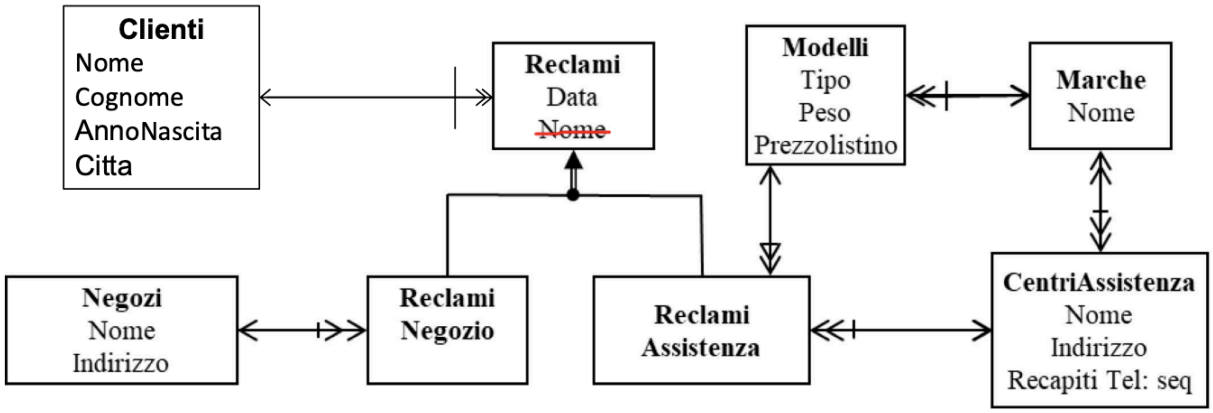
Esercizio: Catena Elettrodomestici

Una catena di negozi vende elettrodomestici e mantiene informazioni sui relativi centri di assistenza. Per ogni modello di elettrodomestico interessa il tipo, il peso, il prezzo di listino, e la marca. Per ogni marca di elettrodomestico interessa il nome e interessa conoscere i relativi centri di assistenza. Per ogni marca ci possono essere numerosi modelli di elettrodomestici.

Di un centro d'assistenza interessano il nome, l'indirizzo, un insieme di recapiti telefonici, e l'insieme di marche per le quali offre assistenza.

La catena tiene inoltre traccia di eventuali reclami ricevuti. Per ogni reclamo interessano la data e il nome indicato dal segnalante. Se il reclamo riguarda un negozio, interessa inoltre conoscere il negozio in questione – per ogni negozio, la catena tiene traccia di un nome, un indirizzo, e l'insieme dei reclami ricevuti.

Se invece riguarda un'operazione di assistenza, interessa tenere traccia del modello di elettrodomestico in questione e del centro di assistenza che ha gestito la riparazione. Il centro di assistenza deve essere uno di quelli che offrono assistenza per la marca del modello di elettrodomestico



Conoscenza Astratta

Riguarda i fatti generali che descrivono:

1. **Struttura della conoscenza concreta**
   - Collezioni
   - Tipi di entità
   - Associazioni
2. **Restrizioni sui valori e sull'evoluzione dei dati**
   - Vincoli d'integrità
3. **Tipologie di vincoli**
   - **Vincoli statici**: limitano i valori ammessi in un dato momento (es. un'età non può essere negativa).
   - **Vincoli dinamici**: regolano le modalità di aggiornamento o evoluzione dei dati nel tempo (es. un conto corrente non può andare in rosso).
4. **Regole di derivazione**

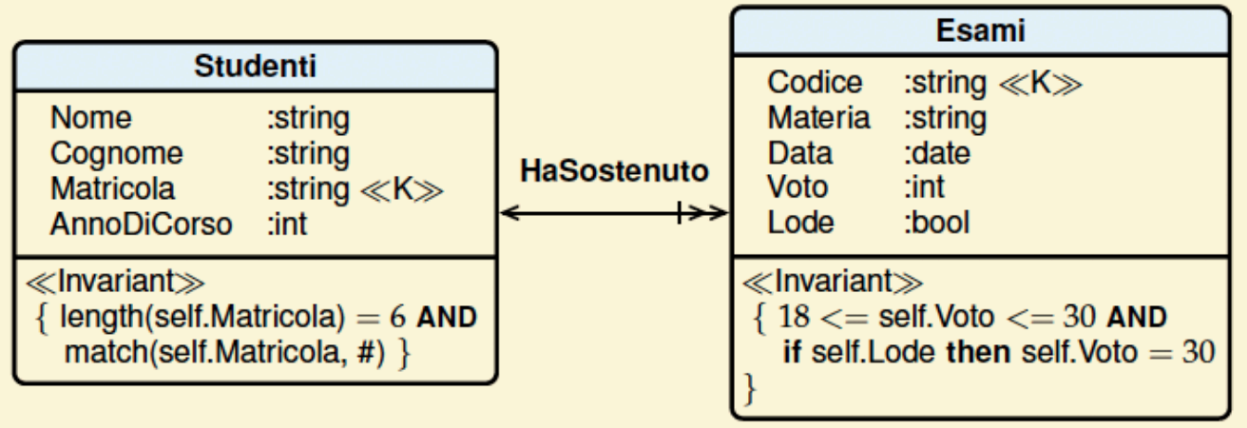
- Permettono di calcolare nuovi fatti a partire da quelli già noti (es. calcolare l'età a partire dalla data di nascita).

Vincoli di Integrità

- **Definizione**: restrizioni sui valori possibili della conoscenza concreta, necessarie oltre alla semplice definizione della struttura dei dati.
- **Scopo**: garantire coerenza, correttezza e affidabilità dei dati nella base di dati.
- **Nota**: non basta definire collezioni, tipi di entità e associazioni; i vincoli d'integrità specificano condizioni che i dati devono sempre rispettare.

Descrittore di Classe con Vincoli

- La **definizione di una classe** può includere vincoli.
- **Chiavi**: attributi marcati con la stringa `K` rappresentano le chiavi della classe, ossia gli identificatori univoci degli oggetti appartenenti alla classe.



Esempi di Vincoli di Integrità

- **Vincolo di appartenenza**:

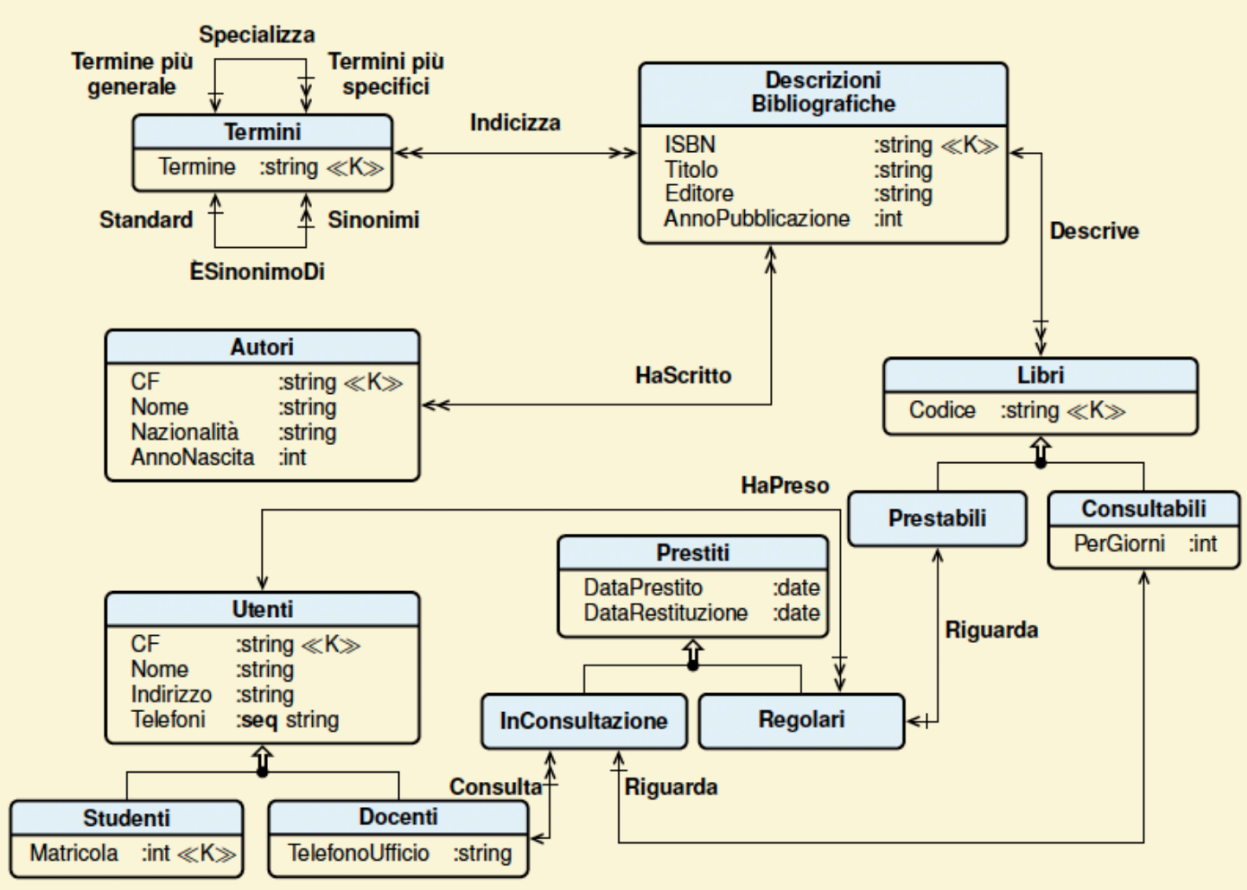
  Un impiegato può essere nominato direttore **solo** del dipartimento a cui appartiene.

- **Vincolo gerarchico**:

  Nessun impiegato può percepire uno stipendio **superiore a quello del proprio direttore**.

- **Vincolo geografico**:

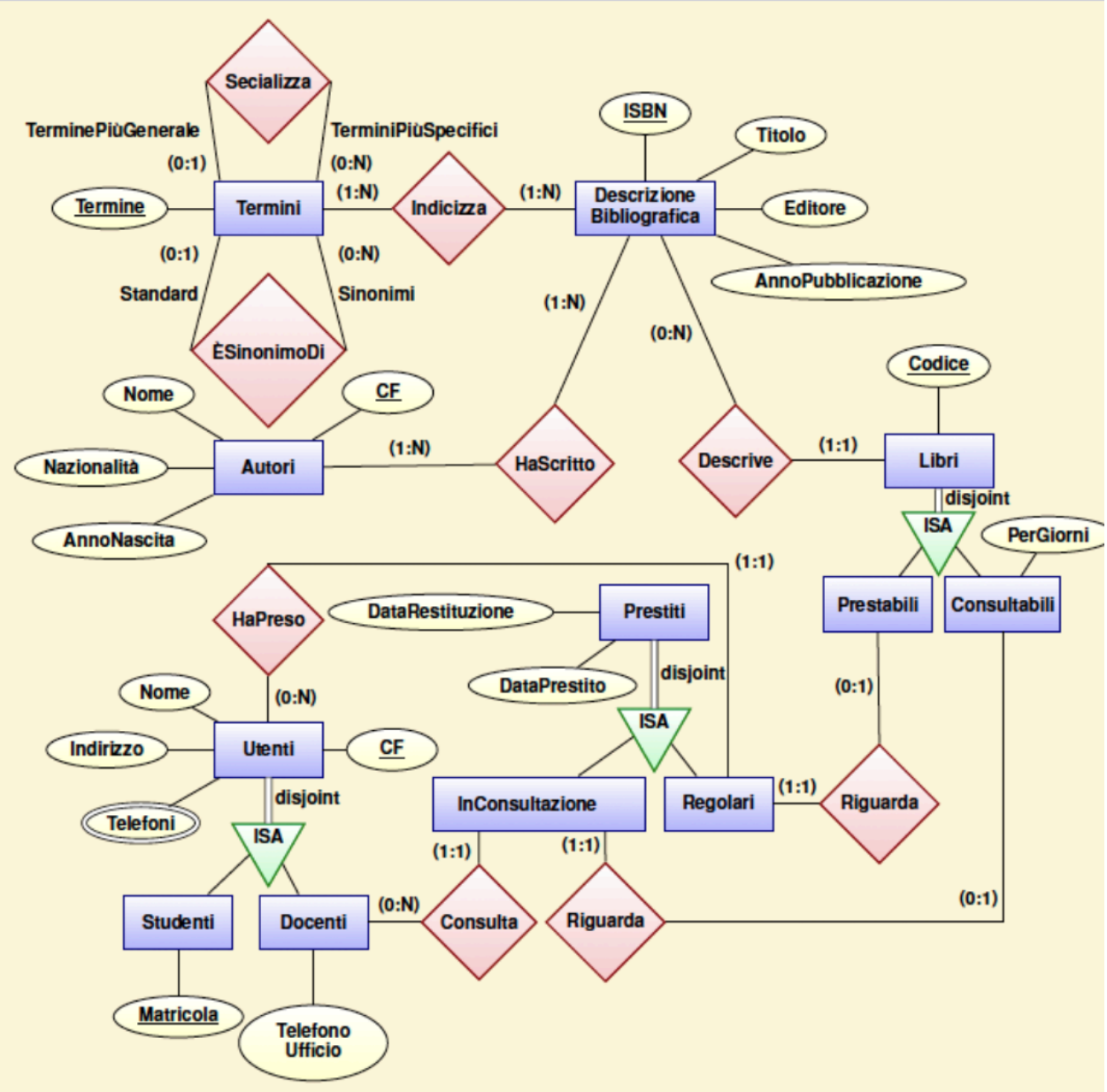
  Ogni impiegato deve avere la residenza nella **stessa città** in cui ha sede il dipartimento in cui lavora.



Modello Entità-Relazione (ER) – Classico

- È il **modello più diffuso** per la progettazione concettuale di basi di dati.
- **Limitazioni**:
  - Non gestisce **gerarchie di inclusione** tra collezioni.

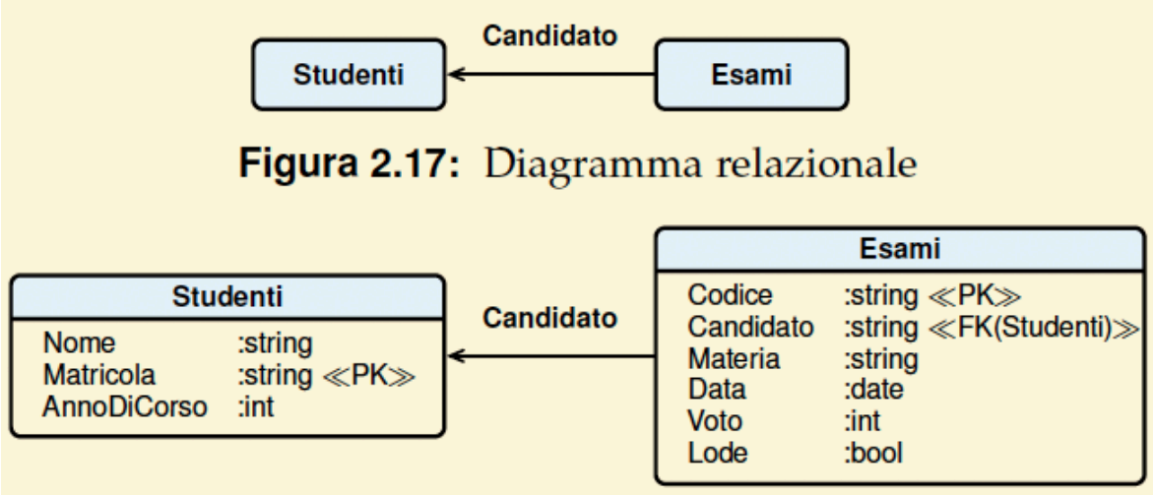
    - Non distingue tra **collezioni** e **tipi**.
    - Non supporta **ereditarietà**.
- **Caratteristiche principali**:
    - Definisce un meccanismo per modellare direttamente le **associazioni** (anche non binarie o con proprietà).
    - Fornisce due meccanismi di astrazione:
        1. Modellare insiemi di **entità** e le loro **proprietà**.
        2. Modellare le **associazioni** (chiamate *relazioni*).
    - Le collezioni sono dette **tipi di entità**.
    - Gli attributi possono assumere **solo valori primitivi**.



Modello Relazionale

- È il **modello dei dati utilizzato nei sistemi commerciali** (DBMS relazionali).
- **Costrutti fondamentali**:
  - **Ennupla**: insieme di coppie *(attributo, tipo primitivo)*.
    - Un valore di tipo ennupla = insieme di coppie *(attributo, valore)* (campi).
  - **Relazione**: insieme di ennuple dello stesso tipo.

- **Chiavi**:
  - **Superchiave**: insieme di attributi che identificano univocamente un'ennupla di una relazione.
  - **Chiave**: superchiave minima.
  - **Chiave primaria**: chiave scelta come identificatore principale.
- **Associazioni**: rappresentate tramite **chiavi esterne**, che assumono valori della chiave primaria di un'altra relazione.



Analisi dei requisiti - Fasi principali

1. **Analizzare il sistema esistente**
   - Studiare come funzionano i processi attuali (manuali o informatizzati).
   - Identificare i problemi, le inefficienze e i punti da migliorare.
2. **Raccogliere i requisiti informali**
   - Interviste, questionari, osservazioni, documentazione esistente.
   - In questa fase i requisiti sono ancora espressi in linguaggio naturale, quindi possono essere ambigui o ridondanti.
3. **Eliminare ambiguità, imprecisioni e disuniformità**

- Rendere i requisiti **chiari, coerenti e non contraddittori**.
- Formalizzare il più possibile il linguaggio.

4. **Raggruppare i requisiti**
   - Separare ciò che riguarda **dati**, **vincoli** e **operazioni**:
     - *Dati*: quali entità e attributi devono essere rappresentati.
     - *Vincoli*: regole di integrità, cardinalità, obbligatorietà.
     - *Operazioni*: query, aggiornamenti, report, ecc.
5. **Costruire un glossario**
   - Definire chiaramente i termini usati, evitando sinonimi e ambiguità.
   - Serve da riferimento unico per progettisti e committenti.
6. **Disegnare lo schema di settore**
   - Una prima bozza concettuale dello schema ER che rappresenta solo una parte (settore) del sistema, utile per validare i concetti raccolti.
7. **Specificare le operazioni**
   - Indicare in modo dettagliato le interrogazioni e le modifiche che il sistema deve supportare.
8. **Verificare la coerenza tra operazioni e dati**
   - Assicurarsi che i dati raccolti e modellati siano sufficienti a supportare tutte le operazioni richieste.
   - Se non lo sono, bisogna tornare indietro e aggiornare il modello dei dati.

Documentazione Descrittiva – Linee guida

Il **linguaggio naturale** (italiano, inglese, ecc.) è utile per esprimere requisiti, ma è **ambiguo**: una frase può avere più interpretazioni, generando errori di progettazione.

Per ridurre questi problemi si seguono alcune regole:

1. **Studiare e comprendere il sistema informativo**

- Analizzare bene l'organizzazione e le necessità informative di tutti i reparti/settori.
- Coinvolgere gli utenti finali per non trascurare aspetti pratici.

2. **Scegliere il corretto livello di astrazione**
   - Non scendere subito nei dettagli tecnici.
   - Restare al livello concettuale, parlando di entità, relazioni, funzioni e non di implementazione.
3. **Standardizzare la struttura delle frasi**
   - Usare schemi fissi, ad esempio:
     - *"L'entità $[X]$ ha attributo $[Y]$ di tipo $[Z]$"*
     - *"Il sistema deve permettere a [utente] di [operazione] su [entità]"*
   - In questo modo si riducono fraintendimenti.
4. **Suddividere le frasi articolate**
   - Evitare periodi lunghi e complessi.
   - Meglio frasi brevi e dirette, ognuna con un solo concetto.
5. **Separare i requisiti sui dati da quelli sulle funzioni**
   - *Requisiti sui dati*: quali entità, attributi, relazioni e vincoli devono esistere.
   - *Requisiti sulle funzioni*: quali operazioni/interrogazioni devono essere possibili.

Organizzazione di Termini e Concetti

Dopo la raccolta dei requisiti, bisogna renderli **chiari, coerenti e strutturati**.

I passi principali sono:

1. **Eliminare ambiguità, imprecisioni e disuniformità**
   - Identificare **omonimi** (stessa parola con significati diversi) ed evitare confusioni.

    - Esempio: “sede” può significare sia “ufficio” che “città”, va chiarito.
- Identificare **sinonimi** (parole diverse con lo stesso significato) e sceglierne uno solo.
    - Esempio: “studente” e “allievo” → uniformare in “studente”.
- Rendere **espliciti i riferimenti** tra termini collegati.
    - Esempio: “corso” fa riferimento a una “materia” e a un “docente”.

2. **Riorganizzare le frasi per concetti**
    - Separare i requisiti in **categorie**:
        - **Dati** → entità, attributi, relazioni (es. *Uno studente ha una matricola*).
        - **Vincoli** → restrizioni (es. *Ogni corso deve avere un solo docente titolare*).
        - **Operazioni** → funzionalità richieste (es. *Il sistema deve permettere di iscrivere uno studente a un corso*).
3. **Costruire un glossario dei termini**
    - Elenco dei concetti usati con definizioni univoche e condivise.
    - Esempio:
        - *Studente*: persona iscritta all’università.
        - *Corso*: attività didattica con un docente e più studenti.
4. **Disegnare lo schema**
    - Tradurre dati e relazioni in un primo **schema concettuale** (es. diagramma E-R).
5. **Specificare le operazioni**
    - Indicare chiaramente cosa deve poter fare il sistema (inserimento, cancellazione, ricerca, aggiornamento, report, ecc.).
6. **Verificare la coerenza tra operazioni e dati**
    - Controllare che ogni operazione faccia riferimento a dati e relazioni realmente presenti nello schema.

- Se un’operazione richiede dati non ancora previsti, lo schema va corretto.